EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A while loop always executes at least once.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

DO NOW — RECALL

Iteration in C#

Name: _____ Class: _____ Date: _____

1. What is the difference between = and == in C#?

2. Give one reason Unicode may be chosen instead of standard ASCII.

3. Convert 01011000 0011 to denary using an 8-bit two’s-complement mantissa and a 4-bit two’s-complement exponent.

4. mark=85. The selection is: if mark >= 80 then print("Distinction"); else if mark >= 50 then print("Pass"); else print("Fail"). Determine the output and explain which branch runs.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

= assigns a value. == compares two values for equality.

2. Model answer

Unicode supports a much larger range of characters and writing systems.

3. Model answer

5.5 Mantissa 01011000 = 0.6875 ; exponent 0011 = 3 ; $0.6875 \times 2^3 = 5.5$.

4. Model answer

Distinction. The first condition is true, so its branch runs and the later branches are skipped.
Conditions are tested top to bottom. The first true branch runs, so the later branch is skipped.

By the end, you can…

• explain iteration and distinguish count-controlled from condition-controlled repetition;

• write and trace for , while and do...while loops;

• choose a suitable loop and test it in .NET Fiddle.

Vocabulary — know these terms

Recall the term — Iteration

Recall the term — Count-controlled iteration

Recall the term — Condition-controlled iteration

Recall the term — Counter

Recall the term — for loop

Recall the term — while loop

Recall the term — do...while

Recall the term — Condition

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Validated age
Keep asking for an age until the user enters a value from 0 to 120 inclusive.
When valid, output Accepted .

Question 4

Password attempts
Use do...while to keep asking until the password is "computer" .
Count how many attempts were made and output the total after the correct password.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Countdown
Use a for loop to output:
10, 9, 8, ... 1

Question 2

Times-table generator
Ask for a whole number and output its first 10 multiples.

NEW LEARNING

Read, annotate and explain

Three loop patterns

```
for (int i = 0; i < 5; i++)
{
// repeat
}

while (condition)
{
// repeat
}

do
{
// repeat
}
while (condition);
```

Choose by purpose: known number of repetitions → for . Unknown number, test first → while . Must run at least once → do...while .

Iteration repeats instructions

Iteration means executing a block of instructions repeatedly. The loop needs a rule that determines whether another repetition happens.

for = count-controlled

The header sets the counter, tests whether the loop continues, then changes the counter after each repetition.

while = test before repeating

A while loop is useful when you do not know how many attempts will be needed.

If the condition is false before the first test, the loop can execute zero times .

do...while = run first, test after

The body executes before the condition is checked, so it always runs at least once.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Build a times-table program

Read the number

```
Console.Write("Number: ");
int number =
    int.Parse(Console.ReadLine());
```

This happens once before the repetition begins.

Count from 1 to 5

```
for (int i = 1; i <= 5; i++)
{
    // repeated instruction
}
```

i = 1 starts the counter; i <= 5 keeps the loop running; i++ adds 1 after each repetition.

Calculate each multiple

```
for (int i = 1; i <= 5; i++)
{
    int answer = number * i;
    Console.WriteLine(answer);
}
```

The same two instructions run five times, but i changes each time.

Trace a known input

```
number = 7

i = 1 → 7
i = 2 → 14
i = 3 → 21
i = 4 → 28
i = 5 → 35
```

Repeat the method with different inputs in your notebook. Check every intermediate value.

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Output the squares 1² to 5²

Use one for loop. Each repetition should output the square of the current counter.

```
for (int i = ?; i <= ?; i++)
{
    Console.WriteLine(?);
}
```

Method / working:

2. Validate 1 to 5

Keep asking until the user enters a number from 1 to 5 inclusive.

Method / working:

3. Password with do...while

Ask at least once, then continue until the answer is "computer" .

Method / working:

4. Describe iteration precisely

An OCR question asks how iteration is used in a function. A strong answer refers to the specific loop , the repeated lines and the condition that keeps it running.

```
while (number > 0)
{
    number = number / 2;
}
```

Method / working: